Postfix Evaluation –

Space Complexity Analysis

Program:

```
int postfixEvaluation(char *postfix)
{
    Stack st;
    create(&st, strlen(postfix));

    int i = 0;
    int r, op1, op2;

    while (postfix[i] != '\0')
    {
        if (isOperand(postfix[i]))
        {
            push(&st, postfix[i] - '0');
        }
        else
        {
            op2 = pop(&st);
            op1 = pop(&st);

            switch (postfix[i])
            {
            case '+':
                r = op1 + op2;
                break;
            case '-':
                r = op1 - op2;
                break;
            case '*':
                r = op1 * op2;
                break;
            case '/':
                r = op1 / op2;
```

```
                break;
            case '%':
                r = op1 % op2;
                break;
            case '^':
                r = pow(op1, op2);
                break;
            default:
                cout << "Invalid Operator" << endl;
                return 0;
            }
            push(&st, r);
        }
        i++;
    }
    return pop(&st);
}
```

Space Complexity Analysis

Let $\boldsymbol{n}$ *be the length of the* $postfix$ *string* $(i.e., strlen(postfix))$.

1. $\boldsymbol{Input\ Space\ Complexity}$: $\boldsymbol{O(n)}$

- $\boldsymbol{What\ it\ is}$: *The space occupied by the input data itself.*
- $\boldsymbol{Why}$: *Postfix function takes a character pointer* $char\ *postfix$ *as its argument. The string being passed into this function consists of* n *characters, which inherently takes up* $O(n)$ *space in memory.*

2. $\boldsymbol{Auxiliary\ Space\ Complexity}$: $\boldsymbol{O(n)}$

- $\boldsymbol{What\ it\ is}$: *The extra, temporary space used by the algorithm during its execution (excluding the input).*

- $\boldsymbol{Why}$: *The auxiliary space is entirely dominated by the stack created on this line:*

  $\boldsymbol{create(\&st, strlen(postfix))};$

  As it is allocating a stack with a maximum capacity equal to the length of the postfix expression (n)*, the memory allocated for the stack's internal array is proportional to* n*. Even though the stack might not get completely full in a valid postfix expression (it usually holds operands temporarily), the allocated space is strictly* $O(n)$*. The other variables* $(i, r, op1, op2)$ *only*

$take\ a\ constant\ O(1)\ amount\ of\ space.$

$\boldsymbol{Total\ Space\ Complexity}$: $O(n)$

$(Total\ Space\ =\ Input\ Space\ +\ Auxiliary\ Space)$